

**MID-TERM FINAL YEAR PROJECT
PROGRESS REPORT**

Edge AI Integrated PIR Based Smart Home Security System Using IoT

Submitted for the Mid-Term Final Year Project Evaluation
B.Tech in Electronics and Communication Engineering
7th Semester | Batch 2023-27

NAME	UNIVERSITY ROLL NO.
SHAYAN BASAK	16900323120
SHANTANU MONDAL	16900323119
SHOURJYA NANDI	16900323122
SHIBAM DEBNATH	16900323121
SAURAV GHOSH	16900323113

**Under the Guidance of
PROF. SUBHAM PRAMANIK**

Assistant Professor
Department of Electronics and Communication Engineering



Academy of Technology
Aedconagar, Hooghly – 712121, West Bengal, India

ABSTRACT

This report presents the ongoing development of an edge-AI-enabled smart home security system using an **ESP32-S3 microcontroller**. The project extends the earlier PIR-based IoT security concept by moving intelligent image analysis toward the edge. The current work has successfully verified ESP32-S3 operation, 8 MB **PSRAM availability**, a 6 MB tensor arena allocation, model loading and TensorFlow Lite Micro interpreter creation. It is intended to provide local human verification after a motion event, thereby helping **distinguish a human presence** from other motion sources and reducing unnecessary alerts. The project therefore combines conventional sensing, IoT communication and embedded artificial intelligence while keeping PIR sensing and IoT-based alerting as the core security functions.

CONTENTS

No.	Topics	Page
1.	Introduction	3
2.	Literature Review and Identified Gap	4
3.	Methodology and System Design	4
4.	Implementation Progress	6
5.	Results and Analysis	7
6.	Conclusion and Future Scope	8
7.	References	9

1. INTRODUCTION

Home security systems increasingly combine sensing, embedded processing and communication to detect suspicious activity and notify users. The earlier version of this project was based on a PIR sensor, camera, LDR, GSM/IoT communication and alarm mechanism. Its main objective was to avoid continuous surveillance and to trigger detailed verification only when motion was detected.

The present stage of the project develops the intelligent decision layer further by using edge artificial intelligence. Instead of depending completely on cloud processing, the image-processing model is intended to run locally on an ESP32-S3-class embedded platform. This approach is intended to reduce communication dependency and provide a faster local response while keeping the system compact and suitable for low-cost embedded deployment.

1.1 Background and Project Evolution

The Phase-I project report established a PIR-based smart home security architecture. The earlier report identified false triggering, continuous monitoring, power consumption, delayed response and lack of an intelligent decision layer as important limitations. The present work continues that direction by introducing a TinyML/edge-inference stage for human detection.

The development has therefore progressed from a sensor-triggered security concept toward an embedded intelligent security system. The current emphasis is not yet on claiming final detection accuracy; it is on establishing a reliable embedded inference platform and making the selected neural-network model compatible with TensorFlow Lite Micro.

1.2 Problem Statement

Traditional low-cost motion-triggered security systems can generate false alarms because motion alone does not establish that a human is present. Cloud-based visual verification can improve intelligence but introduces network dependency, latency and additional infrastructure. The project therefore aims to develop a local edge-AI decision layer capable of analysing camera input on an ESP32-S3 platform after a motion event.

1.3 Objectives

- Develop a low-cost embedded smart security system using the existing PIR-triggered concept as the first-stage detector.
- Use an ESP32-S3 platform for local image-processing and AI inference.
- Integrate a TensorFlow Lite Micro model into the embedded firmware.
- Use available PSRAM efficiently for the neural-network tensor arena.
- Confirm human presence locally before activating the final alert mechanism.
- Reduce unnecessary cloud dependence and improve the potential response time.
- Validate the complete pipeline through measured embedded experiments before reporting final accuracy or latency.

2. LITERATURE REVIEW AND IDENTIFIED GAP

The earlier project report reviewed PIR-based intrusion detection, camera-assisted security systems, IoT monitoring and energy-efficient surveillance. Prior work showed that PIR sensing is useful for low-cost motion detection but can suffer from environmental sensitivity and false triggering. Camera-based verification improves confidence but increases computational and power requirements. Cloud-connected systems provide remote accessibility but can depend heavily on internet connectivity.

The gap identified for the present stage is the absence of a compact local intelligent verification layer that can operate directly on an embedded microcontroller. **Edge AI addresses** this gap by placing the inference step close to the sensor rather than sending every image to a remote server.

Existing Approach	Limitation	Present Project Direction
PIR-only detection	Motion does not confirm a human; false alarms possible.	Use AI-based visual verification after a trigger.
Continuous CCTV	Higher storage and power requirements.	Activate intelligent processing when required.
Cloud AI	Network dependency and additional latency.	Run inference locally at the edge.
Large desktop/server model	Not suitable for constrained embedded memory.	Use a microcontroller-compatible TensorFlow Lite model.

3. METHODOLOGY AND SYSTEM DESIGN

3.1 Proposed System

The proposed system follows a multi-stage security workflow. A PIR sensor can act as a low-power event trigger. When motion is detected, the camera supplies an image frame to the ESP32-S3. The embedded AI model processes the frame and determines whether the visual content satisfies the human-detection criterion. A confirmed detection can then activate the alarm and communication subsystem.

[System block diagram – PIR Sensor → Camera → ESP32-S3 Edge AI → Human Detection → Human activity classification → Alarm/GSM/IoT Alert]

3.2 Component Description

Component	Function	Role in Project
ESP32-S3	Embedded controller	Controls the system and supports the Edge AI enhancement
PIR sensor	Motion detection	First-stage event trigger
Camera module	Image capture	Provides visual evidence after motion
LDR	Light measurement	Supports illumination/light-condition decision
Buzzer/Siren	Local alarm	Provides immediate local warning
GSM/IoT module	Remote communication	Sends security notification to the user
8 MB PSRAM	External memory	Supports large buffers and AI tensor workspace

3.3 Edge AI Inference Pipeline

- Capture an input image after the first-stage trigger.
- Pre-process the image to the dimensions and numerical format expected by the neural-network model.
- Copy the processed input into the TensorFlow Lite Micro input tensor.
- Execute the interpreter on the ESP32-S3.
- Read the output tensor and apply the human-detection decision rule.
- Generate the security response only after the AI decision confirms the required condition.

3.4 Software Architecture

The firmware is written in C/C++ for the ESP32-S3 environment. The current AI integration uses TensorFlow Lite Micro components, including the model data header, Micro Interpreter and Micro Mutable Op Resolver. A statically allocated tensor arena is used to provide memory for intermediate tensors and buffers. PSRAM is used to make a substantially larger arena available than would be practical with internal RAM alone.

4. IMPLEMENTATION PROGRESS

4.1 ESP32-S3 Hardware Bring-Up

The first major implementation step was establishing a reliable ESP32-S3 development environment and verifying that the target board was correctly detected and programmed. This step confirmed that the embedded platform itself is functional before introducing the computational load of neural-network inference.

The ESP32-S3 was successfully detected and the firmware could execute on the target. This provides the base platform for the subsequent camera and TinyML stages.

4.2 PSRAM Verification

The board was verified to have 8 MB of PSRAM. This is particularly important because image buffers and neural-network intermediate tensors can require substantially more memory than ordinary embedded applications.

[Serial-monitor evidence – ESP32-S3 detected and 8 MB PSRAM reported]

4.3 Tensor Arena Allocation

A tensor arena of approximately 6 MB was successfully allocated for TensorFlow Lite Micro. This is a significant implementation milestone because it confirms that the project can reserve a memory region large enough to attempt execution of the selected model on the ESP32-S3 platform.

[Serial-monitor evidence – 6 MB tensor arena allocation]

4.4 TensorFlow Lite Micro Integration

The embedded model was successfully included through a generated model-data header. The TensorFlow Lite Micro model object could be obtained and the interpreter could be created. These steps confirm that the model binary is being correctly linked into the firmware and that the runtime can construct the interpreter object.

4.5 Current Technical Limitation

The remaining failure occurs at `AllocateTensors()`. The earlier model contains a 5-dimensional `STRIDED_SLICE` operation. The current TensorFlow Lite Micro operator implementation/resolver used by the project does not successfully allocate this graph. Therefore, the current milestone should be described as “model loaded and interpreter created, but tensor allocation not yet successful,” rather than as successful AI inference.

This distinction is important for the evaluation report because no final detection accuracy, inference time or real-time human-detection claim should be made until `AllocateTensors()` completes and the model has been exercised with actual camera data.

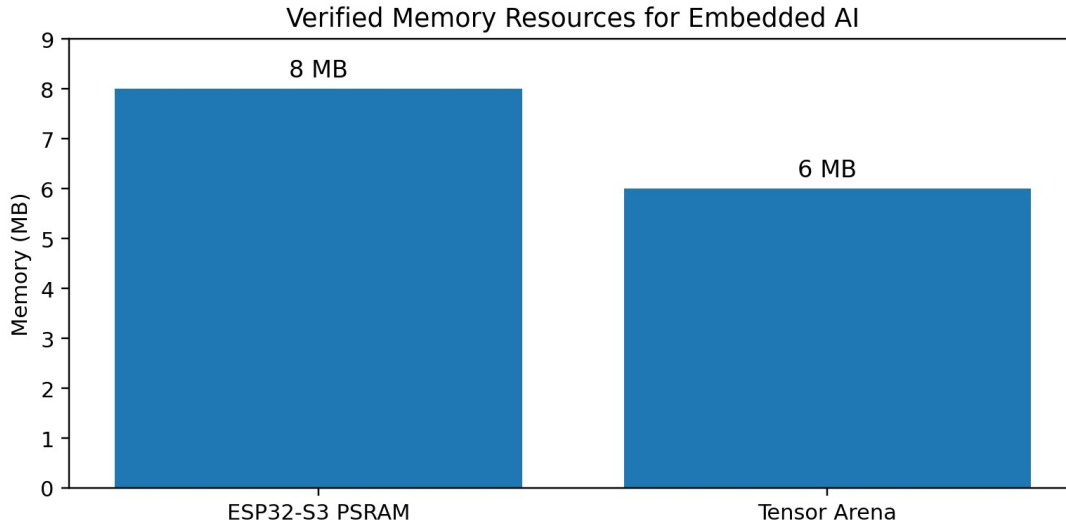
4.6 Planned Resolution

- Inspect the original model graph and identify the exact `STRIDED_SLICE` node and its input/output tensor dimensions.
- Determine whether the model can be converted to a TensorFlow Lite Micro-compatible graph without the problematic 5-D operation.
- If necessary, modify or retrain the model architecture so that unsupported graph operations are avoided.
- Regenerate the C/C++ model-data array and repeat interpreter and tensor-allocation tests.
- Validate input tensor shape, quantization/data type and output tensor interpretation.
- Connect the validated inference pipeline to camera capture and measure inference latency.
- Complete the security decision and alert stages after AI inference is stable.

5. RESULTS AND ANALYSIS

5.1 Verified Milestones

Milestone	Status	Interpretation
ESP32-S3 detection and firmware execution	COMPLETED	Target hardware and development environment are functional.
8 MB PSRAM verification	COMPLETED	External memory is available for memory-intensive embedded AI work.
6 MB tensor arena allocation	COMPLETED	A large TensorFlow Lite Micro memory arena can be reserved.
Model loading	COMPLETED	Embedded model data is accessible by the firmware.
Interpreter creation	COMPLETED	TensorFlow Lite Micro interpreter object is constructed.
<code>AllocateTensors()</code>	PENDING	Blocked by the model's 5-D <code>STRIDED_SLICE</code> operation.
Actual model inference	PENDING	Cannot be claimed until tensor allocation succeeds.
Camera-to-AI security decision	PENDING	Requires successful embedded inference.



Note: Tensor arena is an allocation within the available memory; values are not additive.

5.2 VERIFIED MEMORY RESOURCE PLOT

Figure 5.1: Verified memory resources for the embedded AI implementation

The graph presents the memory figures verified during implementation: 8 MB of PSRAM was available on the ESP32-S3 and approximately 6 MB was allocated as the TensorFlow Lite Micro tensor arena. The tensor arena is an allocation within the available memory, so the values are not additive. These measurements demonstrate the available memory foundation for the embedded AI workload; final inference performance still depends on successful tensor allocation and model compatibility.

6. CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The project has progressed from a PIR-based smart security concept toward an embedded edge-AI architecture using the ESP32-S3. The current implementation has established the hardware and memory foundation required for TinyML: the ESP32-S3 is operational, 8 MB PSRAM has been verified, a 6 MB tensor arena has been allocated, and the TensorFlow Lite Micro model and interpreter can be initialized. The remaining limitation is a model-compatibility issue at tensor allocation caused by a 5-D STRIDED_SLICE operation. Resolving this issue and completing measured inference experiments are the immediate goals before final system validation.

6.2 Future Scope

- Use a fully TensorFlow Lite Micro-compatible lightweight model for embedded human detection.
- Optimize preprocessing and inference for lower latency and memory usage.
- Integrate live camera capture with the AI inference pipeline.
- Add confidence-based decision logic to reduce false alarms.

- Complete GSM/IoT and local alarm integration.
- Measure power consumption and investigate low-power operating modes.
- Evaluate model performance across different lighting and indoor conditions.
- Explore model quantization and further hardware acceleration where practical.

7. REFERENCES

- [1] Surantha, N. (2018). Smart Home Security System Using Object Detection. ScienceDirect.
- [2] Sahoo, K. C. (2017). IoT Based Intrusion Detection System Using PIR Sensor.
- [3] Netinant et al. (2024). IoT Driven Home Security System. MDPI.
- [4] IoT Based Home Security System. IJARCCCE Journal.
- [5] Smart Home Security Using PIR and Camera. IJERT.
- [6] TensorFlow Lite for Microcontrollers documentation and examples. Google/AI Edge.
- [7] Espressif Systems. ESP32-S3 Series Technical Reference and development documentation.

Project Guide Signature: _____ Date: _____